

Drishti

A fully offline visual assistant for blind and low-vision users, running entirely on the Snapdragon Hexagon NPU.

Rachit · rachit2413078@akgec.ac.in · github.com/Rachit0910 · Solo submission, newly created for this Challenge

What it is

Drishti turns a Snapdragon-powered HP PC into an assistive device. Using only the built-in camera, it continuously tells a blind user what is around them, how far away it is, and which way it lies — and reads any text in view on request. It runs at 30 frames per second, on battery, with no internet connection and no cloud inference of any kind.

Assistive perception is one of the few consumer AI workloads where on-device inference is a precondition rather than an optimisation. A cloud round trip of two to three seconds is the difference between hearing about a kerb before or after stepping off it; connectivity is worst exactly where the tool is needed most; and continuous capture-and-upload at 30 fps is not viable on battery. Each of those is a direct argument for a dedicated neural accelerator, which is why Drishti is built for the Hexagon NPU rather than merely ported to it.

The need is large. A systematic review in the National Medical Journal of India puts the pooled prevalence of blindness among Indian adults aged 50 and over at 6.11%, across a sample exceeding 211,000. Existing options are dedicated hardware — expensive, single-purpose, usually imported — or cloud phone apps that upload a photo and wait. Neither offers continuous, real-time awareness that keeps working with no signal.

Technical implementation

Three Qualcomm AI Hub models are chained into one always-on pipeline, all executed through the ONNX Runtime QNN Execution Provider on the Hexagon NPU. Latencies below are AI Hub's published figures on Snapdragon X2 Elite.

Stage	AI Hub model	Precision	Cadence	Latency
Objects & obstacles	YOLOv8-Detection	w8a8	Every frame	0.67 ms
Distance	Midas-V2	w8a8	Every frame	0.62 ms
Scene text	EasyOCR (det + rec)	float	On demand	32.8 ms

The frame budget is the design. At 30 fps a frame must be handled within 33.3 ms. Detection and depth together cost 1.29 ms — 3.9% of that budget. With OCR amortised at 2 Hz the steady state is about 10%, leaving roughly 90% of every frame for capture, the quality gate, post-processing, the salience engine and speech synthesis. That headroom is what makes continuous narration possible at all; the same three models on CPU would consume a large multiple of the budget and a correspondingly larger share of the battery.

- **Snapdragon-specific optimisation.** Finalizing a graph on the Hexagon Tensor Processor is expensive and by default happens on every launch — landing the delay at the worst moment for a blind user. Drishti dumps the compiled QNN context once (ep.context_enable / ep.context_file_path / ep.context_embed_mode) and loads from it thereafter. Because that cost is then paid exactly once, graph finalization also runs at the most aggressive optimisation level, and http_performance_mode is set to burst.
- **Graceful degradation.** Every model loads through one module that walks QNN (NPU) → DirectML (Adreno GPU) → CPU and records which provider accepted the graph. The app logs it at startup and warns explicitly when it is not on the NPU, rather than silently degrading. Identical artifacts in all three cases.
- **Honest measurement.** AI Hub device provisioning compiles and profiles each model on a physical Snapdragon device in Qualcomm's device farm, giving true on-device latency, memory and the per-layer NPU split without owning the hardware. A separate harness measures sustained end-to-end throughput including every non-NPU stage, with a same-

machine CPU baseline. Both numbers are reported, including the gap between them. A float-versus-w8a8 accuracy study reports small-object recall, with the decision rule fixed in advance: if quantization costs more than two points, that model ships at float.

Innovation: the part that is not off-the-shelf

Object detection on an NPU is the reference workflow in Qualcomm's own documentation. The work that makes Drishti usable sits between three off-the-shelf models and something a blind person would keep switched on.

- **Relative depth is not distance.** Midas-V2 predicts relative inverse depth, not metres; announcing it as metres would give confident, wrong distances. Drishti fits the disparity map to metric units every frame using detected objects of known physical size as anchors — a person is a reliable 1.68 m ruler — via a least-squares fit guarded by outlier rejection.
- **Speech is a serial channel.** Piping every detection into a TTS engine produces “chair chair person chair”, which users switch off within a minute. The narration policy scores each observation as hazard × proximity × centrality × confidence, lets imminent hazards interrupt what is being spoken, suppresses facts that have not materially changed, and enforces a hard ceiling on routine utterances — staying silent when the budget is spent.
- **Silence is ambiguous.** Drishti's correct behaviour in an empty corridor is silence. Its behaviour with a thumb over the lens is also silence. A sighted user resolves that by glancing at the camera; the only user this product has cannot. So degraded input is announced, not swallowed — and three conditions are distinguished because the remedy differs: “Camera is covered” (move your hand), “Too dark to see” (turn on a light), “Camera image is blurred” (hold steadier), plus “Camera clear” on recovery so silence becomes meaningful again.

Deployment and accessibility

The primary user cannot see the screen, which inverts the usual design: the audio channel is the interface, and the display is optional scaffolding for a sighted helper during setup.

- **Zero-configuration.** A single signed Windows installer for Windows on Snapdragon, models bundled so first launch works offline. No account, no login, no API key, no configuration wizard — a setup flow that must be navigated visually is one this user cannot complete unaided.
- **Audio-first control.** Spoken output plus a small set of keys findable by touch, each with a short distinct audio confirmation rather than a spoken sentence. Space reads text in view; Escape stops speech immediately; arrows tune narration frequency.
- **Multilingual by architecture.** English and Hindi ship today. No user-facing string appears anywhere in the logic — directions, distance vocabulary, sentence templates and camera messages all live in a phrasebook, and a unit test asserts no English string can leak into a Hindi utterance. Voice selection enumerates installed system voices and matches the locale, falling back to the default with a logged warning rather than failing to speak.
- **Shares the channel.** Blind Windows users already run NVDA or JAWS, which own the audio channel. Drishti keeps utterances short, speaks only above a salience threshold, and yields instantly on Escape. Verifying this alongside NVDA is an explicit item in the user-testing plan, not an assumption.
- **Institutional scale.** Because AI Hub's optimisation targets the Snapdragon compute family rather than one SKU, and the provider chain degrades gracefully, the same installer is expected to run across Snapdragon-powered HP models — making a rollout to a school for the blind or an NGO practical.

What is built and verified today

The repository is not a plan. The following is implemented and passing now, on ordinary hardware, before any Snapdragon device is in hand — a reviewer can clone it and run all 37 tests in under a minute with no camera, no NPU and no account.

Component	Status	Evidence
Narration policy engine	Complete, tested	17 unit tests passing

Component	Status	Evidence
Frame quality gate	Complete, tested	11 unit tests passing
Metric depth calibration	Complete, tested	9 unit tests passing
Localisation (English + Hindi)	Complete, tested	Covered by the policy tests
Provider fallback + context cache	Implemented	Needs a Snapdragon device to validate
Model wrappers, AI Hub export, benchmark harness	Implemented	Needs artifacts / an AI Hub token
OCR recognizer decode	Stubbed	Documented as outstanding

Three defects the tests caught, each of which changed the design: depth jitter across a distance-band boundary re-announced the same obstacle several times a second; one frame mistaking a poster for a person shifted the depth calibration — and therefore every announced distance — by 73%; and darkness and occlusion were being conflated, when they need different tests and produce different advice. All three fixes are in the repository, with the tests that forced them.

Roadmap

- **Phase 1** Export, quantize and profile all three models through AI Hub device provisioning; record real on-device numbers, the NPU layer split, and float-versus-w8a8 accuracy deltas.
- **Phase 2** Wire exported artifacts into the model wrappers; validate the full loop on a Snapdragon-powered HP PC; complete the OCR decode; confirm the context cache removes the cold start.
- **Phase 3** End-to-end benchmarking against a CPU baseline; sustained thermal and power run to substantiate the all-day battery claim.
- **Phase 4** User testing with blind and low-vision participants; native-speaker review of the Hindi phrasebook; NVDA coexistence check; retune the narration policy from the findings.
- **Phase 5** Installer packaging, demonstration video, documentation and the mentor presentation.

Intellectual property and eligibility

- Drishti is a new project, conceived and written solely by the submitter. It shares no code, repository or idea with any team project, hackathon entry or prior work.
- All three models come from the Qualcomm AI Hub model library under their upstream licences: Midas-V2 (MIT), EasyOCR (JaidedAI upstream), YOLOv8-Detection (AGPL-3.0 — noted openly in the repository README, with a permissively-licensed detector identified as the swap should commercial use ever be intended).
- Because the project is newly created for this Challenge, the “significantly modified” requirement for pre-existing proposals does not apply.
- Single submission; no confidential information or trade secrets; written in English; all submitted material is the submitter's own work.

Full technical proposal, architecture diagram and benchmark methodology are in the repository under docs/. Every published figure quoted here is a Qualcomm AI Hub measurement and will be replaced by our own measured value once the profiling jobs have run.